EXP:5 - DYNAMIC PROGRAMMING – KNAPSACK PROBLEM, SUBSET SUM PROBLEM, LONGEST COMMON SUBSEQUENCE PROBLEM

AIM:

1. To maximize the total value of items included in a knapsack without exceeding its capacity, given weights and values of the items.
2. To determine if a subset of a given set of integers sums up to a specified value.
3. To find the length of the longest subsequence common to two sequences, where the subsequence maintains the order of elements.

PROCEDURE:

1. Knapsack Problem:-

Given weights and values of n items, put these items in a knapsack of capacity W to get the maximum total value in the knapsack.

```
def knapsack(weights, values, W):
    n = len(weights)
    dp = [[0 for _ in range(W + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                dp[i][w] = 0
            elif weights[i-1] <= w:
                dp[i][w] = max(values[i-1] + dp[i-1][w-weights[i-1]],
                               dp[i-1][w])
            else:
                dp[i][w] = dp[i-1][w]

    return dp[n][W]

# New data
weights = [2, 3, 4, 5]
values = [6, 5, 8, 9]
```

```
W = 8

print("Maximum value:", knapsack(weights, values, W))
```

Output:

Maximum value: 15

2. Subset Sum Problem:-

Determine if there is a subset of the given set with a sum equal to a given sum.

```
def subset_sum(arr, sum):
    n = len(arr)
    dp = [[False for _ in range(sum + 1)] for _ in range(n + 1)]

    for i in range(n + 1):
        dp[i][0] = True

    for i in range(1, n + 1):
        for s in range(1, sum + 1):
            if arr[i-1] <= s:
                dp[i][s] = dp[i-1][s] or dp[i-1][s-arr[i-1]]
            else:
                dp[i][s] = dp[i-1][s]

    return dp[n][sum]

# New data
arr = [2, 5, 7, 8, 10]
sum = 14

print(subset_sum(arr, sum))
```

Output:

True

3. Longest Common Subsequence Problem:-

Given two sequences, find the length of the longest subsequence present in both of them.

```
def lcs(X, Y):
    m = len(X)
    n = len(Y)
    dp = [[0 for _ in range(n + 1)] for _ in range(m + 1)]

    for i in range(m + 1):
        for j in range(n + 1):
            if i == 0 or j == 0:
                dp[i][j] = 0
            elif X[i-1] == Y[j-1]:
                dp[i][j] = dp[i-1][j-1] + 1
            else:
                dp[i][j] = max(dp[i-1][j], dp[i][j-1])

    return dp[m][n]

# New data
X = "PROGRAMMING"
Y = "PROBLEMING"

print("Length of LCS:", lcs(X, Y))
```

Output:
Length of LCS: 7

Result:

The Knapsack Problem, Subset Sum Problem, and Longest Common Subsequence (LCS) problem were successfully solved using Python. The optimal knapsack value, subset feasibility, and LCS length were obtained respectively.